

Resume ingestion and phase completion

The screenshot shows a VS Code editor with the following components:

- EXPLORER:** Shows the project structure with files like `01_Resume_Ingestion_Phasewise_Impl...`, `Batch Resume Ingestion Workflow.md`, `Resume_RAG_JCE_POT_Beginner_Pro...`, `TS ingestionController.ts`, `src/modules/inges...`, `env`, `TS env.ts`, `TS env.ts src/config`, `RESUME_RAG_V1`, `bulk_resumes`, `node_modules`, `src`, `config`, `TS database.ts`, `TS env.ts`, `TS multerConfig.ts`, `TS skills.ts`, `middleware`, `modules/ingestion`, `controllers`, `TS ingestionController.ts`, `repositories`, `routes`, `services`, `types`, `utils`, `scripts`, `TS app.ts`, `TS servers.ts`, `OUTLINE`, `TIMELINE`, and `VS CODE POKÉMON`.
- EDITOR:** Shows the `TS env.ts` file with the following code:

```
src > config > TS env.ts > env
13 export const env = {
14   resumeCollectionName: process.env.COLLECTION_NAME ?? "resumes",
15   vectorIndexName: process.env.VECTOR_INDEX_NAME ?? "resume_rag_index",
16   mistralApiKey: process.env.MISTRAL_API_KEY ?? "",
17   mistralEmbedModel: process.env.MISTRAL_EMBED_MODEL ?? "mistral-embed",
18   embeddingDimension: parseInt(process.env.EMBEDDING_DIMENSION ?? "1024", 10),
19   useLmParser: process.env.USE_LLM_PARSER === "true",
20   groqApiKey: process.env.GROQ_API_KEY ?? "",
21   groqModel: process.env.GROQ_MODEL ?? "meta-llama/llama-4-scout-17b-16e-instruct",
22   maxUploadSizeMb: parseInt(process.env.MAX_UPLOAD_SIZE_MB ?? "5", 10),
23   resumeIngestionDelayMs: parseInt(process.env.RESUME_INGESTION_DELAY_MS ?? "1000", 10),
24 };
25 export { requireEnv };
```

The **TERMINAL** shows the output of `npm run inges`:

```
(base) jitinjayanandan@Jitins-MacBook-Air resume_rag_v1 % npm run inges
t:bulk
Batch size      : 5
Total batches   : 37
Successfully processed : 180
Failed          : 1
MongoDB Database : Rag_Testcases
MongoDB Collection : resume_rag

Failed resumes:
- Naukri_Amutha[1y_4m].doc (batch 8): Failed to extract text from /Users/jitinjayanandan/33/TestLeaf/genai-assignments/phase-2/week-5 - Sep-29-30/Sun/resume_rag_v1/bulk_resumes/Naukri_Amutha[1y_4m].doc: Unable to read th
is type of file
```
- CHAT:** Shows a chat window with the title `INGESTION PIPELINE PHASED IMPLEMENTATION` and the following text:

Reviewed resume_rag_v1/src

Let's do a final compile check across all touched files:

All changes are ready. Summary of what was added:

 - Configurable inter-resume delay — `RESUME_INGESTION_DELAY_MS` env var (default 1000 ms), also overridable via `--delayMs=` CLI flag. Applied between every resume (not just retries) to smooth out API request pacing and reduce 429 rate-limit errors.
 - Running success/failure tally — after every batch completes, logs `Batch N completed (Success: X, Failed: Y)` and a running total `Processed: A/B`. Success: X, Failed: Y, so counts are never lost even in large runs with heavy log output. The final box-summary still prints totals at the very end.
 - Hardened `ensureFileHashIndex()` — now wrapped in `try/catch` with a `[WARN]` log instead of crashing the whole run if index creation fails (this could explain why the summary was missing previously — an unhandled index error would abort before reaching the summary).
 - Doc updated (`Batch Resume Ingestion Workflow.md`) —

The screenshot shows the MongoDB Cloud Data Explorer interface with the following components:

- ORGANIZATION:** Jitin's Org - 2026-08-15
- PROJECT:** Project 0
- Data Explorer:** Shows the `resume_rag` collection with 180 documents.
- Documents:** Shows a list of documents with columns for `_id`, `fileHash`, `company`, `createdAt`, `email`, `embedding`, `embeddingDimension`, `embeddingModel`, `experienceSummary`, `fileName`, `jobTitles`, `location`, `name`, and `phone`.
- Query:** A query is entered: `{ field: 'value' }` or `Generate query`.
- Results:** Shows a single document with the following fields:

```
{
  "_id": "ObjectID('6a96ca37d367bfcdbf681d2')",
  "fileHash": "fc5ef5747aa3b59a486f364a6057f2ff9a6baeb6c26566ee524a084092c754ae",
  "company": null,
  "createdAt": "2026-09-01T12:51:03.454+00:00",
  "email": "abnerferdinand3@gmail.com",
  "embedding": [1024],
  "embeddingDimension": 1024,
  "embeddingModel": "mistral-embed",
  "experienceSummary": null,
  "fileName": "ABNER RESUME CVV (1).pdf",
  "jobTitles": [],
  "location": null,
  "name": "Problem Solving",
  "phone": "8897218379"
}
```

CodeFileEditSelectionViewGoRunTerminalWindowHelp

resume_rag_v1

Update

EXPLORER

OPEN EDITORS 1 unsaved

RESUME_RAG_V1

src

modules/ingestion

controllers

repositories

ResumeIngestionRepository.ts

routes

ingestionRoutes.ts

services

types

utils

scripts

types

app.ts

server.ts

tests

uploads

env

env.example

.gitignore

01_Resume_Ingestion_Phasewise_Im...

Batch Resume Ingestion Workflow.md

commands

jest.config.js

package-lock.json

package.json

tsconfig.json

OUTLINE

TIMELINE

VS CODE POKÉMON

e_Ingestion_Phasewise_Implementation1.md

commands

1 # command to fully restart

2

3 lsof -ti tcp:3000 | xargs kill -9

4 npm run dev

5

6 for bulk resume ingestion

7 npm run ingest:bulk

8

9 for executing tests

10 npm test

PROBLEMS

OUTPUT

TERMINAL

node

zsh

(base) jitinjayananand@Jitins-MacBook-Air resume_rag_v1 % npm run dev

{ path: ['.env.local', '.env'] }

resume-rag-backend listening on port 3000 (development)

MongoDB connected

CHAT

CLAUDE CODE

INGESTION PIPELINE PHASED IMPLEMENTATION

Phase 16 — Automated Tests: Summary

Test infrastructure:

Installed supertest + @types/supertest (dev dependencies) for HTTP-level integration testing against the real Express app.

Fixed a pre-existing incompatibility: pdf-parse's underlying pdfjs-dist uses dynamic import() for its worker, which fails under Jest's default CommonJS sandbox. Fixed by adding NODE_OPTIONS=--experimental-vm-modules to the test script in package.json .

Fixed uuid@14 (ESM-only package) failing to load under Jest by adding a moduleNameMapper in jest.config.js pointing to a tiny CommonJS shim (uuidShim.js) that wraps crypto.randomUUID() — this only works around a tooling/ESM interop gap, no application logic is mocked.

Unit tests (unit , all pure-function/isolated-dependency tests, no real network/DB calls except where noted):

File	Covers
regex.test.ts	Email/phone/experience regex (pre-existing)
textCleaner.test.ts	cleanResumeText — line breaks, control chars, whitespace, blank lines, symbol preservation
skills.test.ts	detectSkills — case-insensitivity, word-boundary matching, empty input

+ commands

Describe what to build

+ Agent Claude Sonnet 5 High 200K

Local Default Approvals